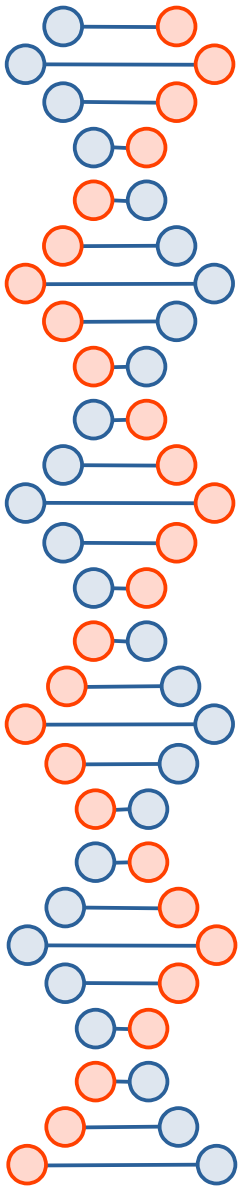
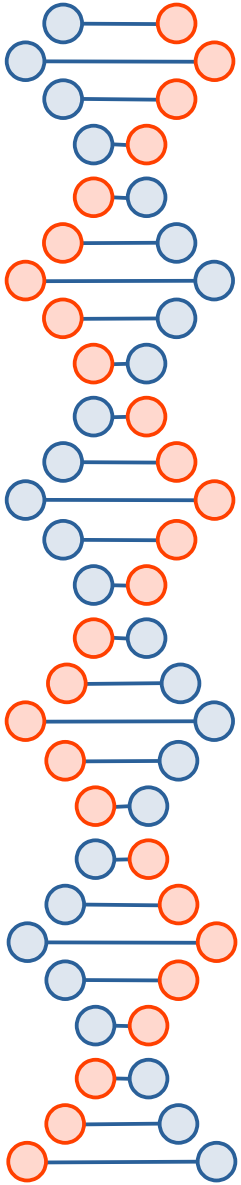


Métricas de evaluación de modelos de clasificación: Exactitud, Precisión, Sensibilidad, Medida F, Matriz de Confusión y curvas ROC

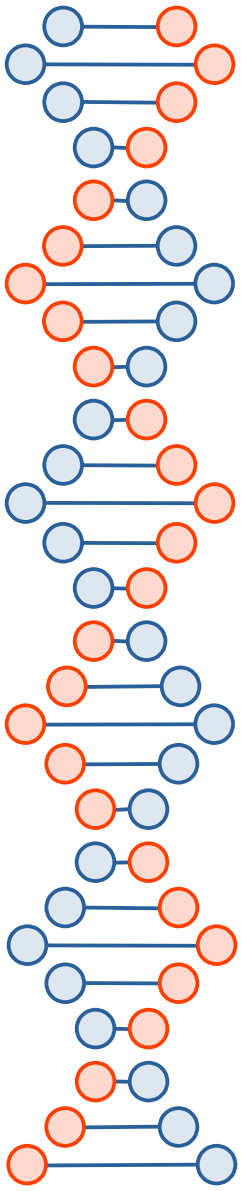
Dr. Josafá Pontes

Machine Learning



Outline

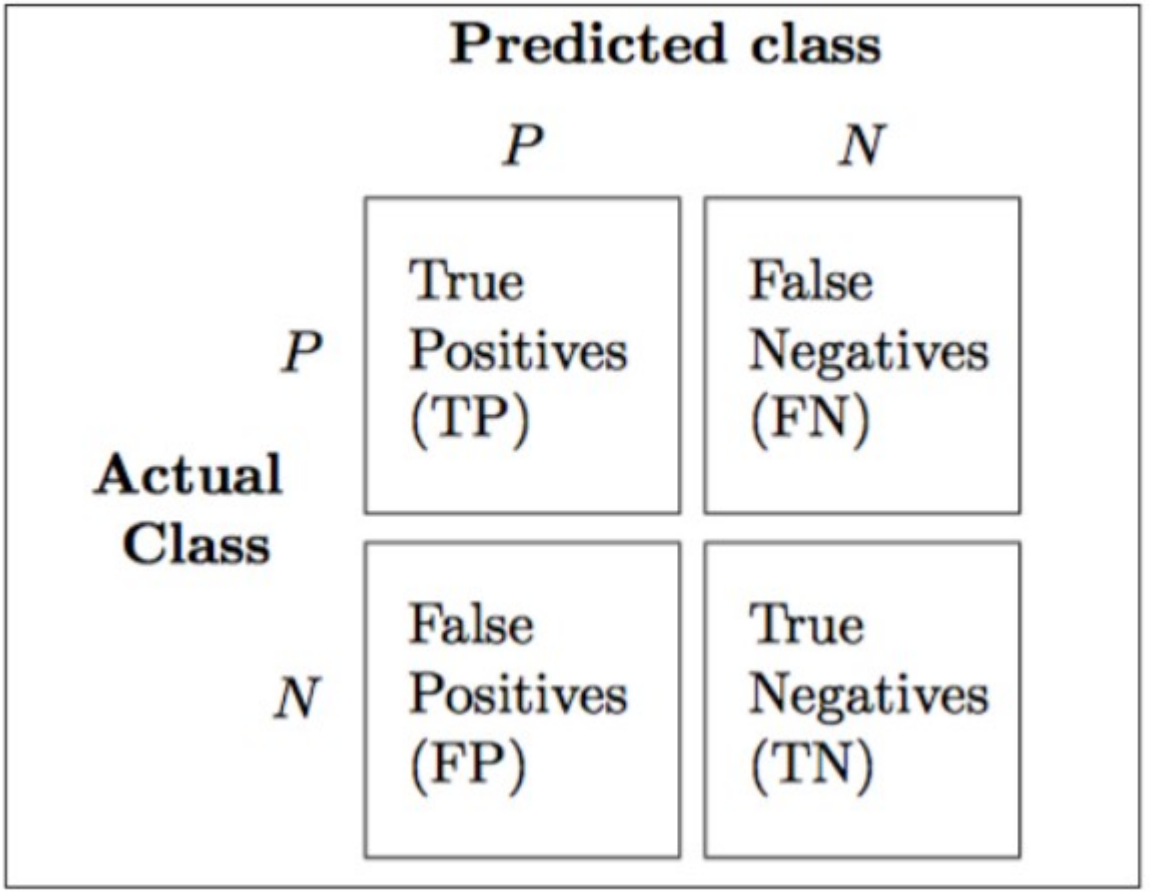
- How to measure classification results.
- How to calculate Accuracy, Precision, Sensitivity, F-Measure.
- How to implement classification accuracy.
- How to implement and interpret a confusion matrix.
- How to interpret a ROC curve.



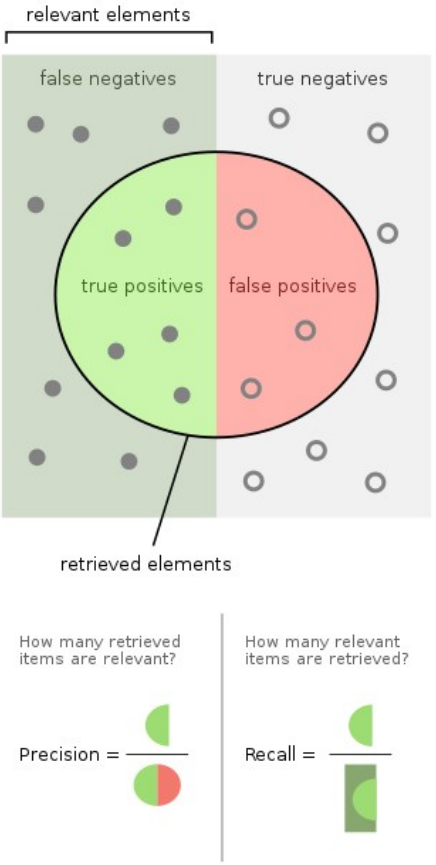
Overview on Evaluation Metrics

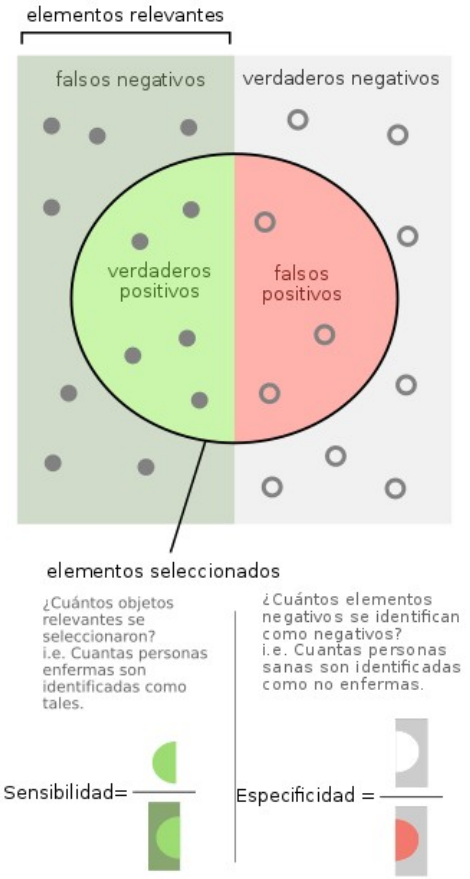
- You must estimate the quality of a set of predictions when training a machine learning model.
- Performance metrics like classification accuracy can give you a clear objective idea of how good a set of predictions is, and in turn how good the model is that generated them.
- This is important as it allows you to tell the difference and select among:
  - Different transforms of the data used to train the same machine learning model.
  - Different machine learning models trained on the same data.
  - Different configurations for a machine learning model trained on the same data.
- As such, performance metrics are a required building block in implementing machine learning algorithms from scratch.

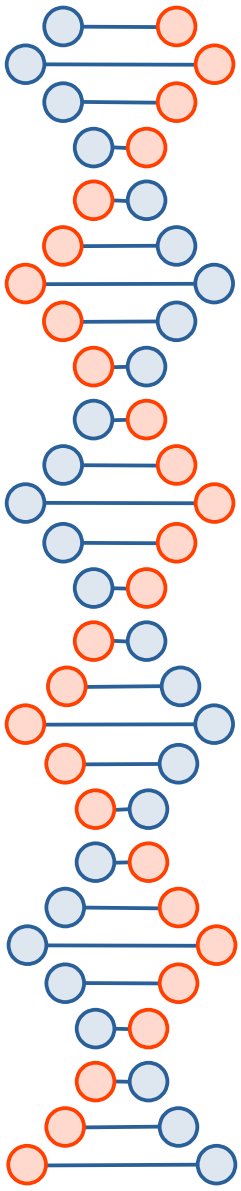
Measuring classification results



Classification performance measurements (1)







Classification performance measurements (2)

$$\text{Precision} = \frac{tp}{tp + fp}$$

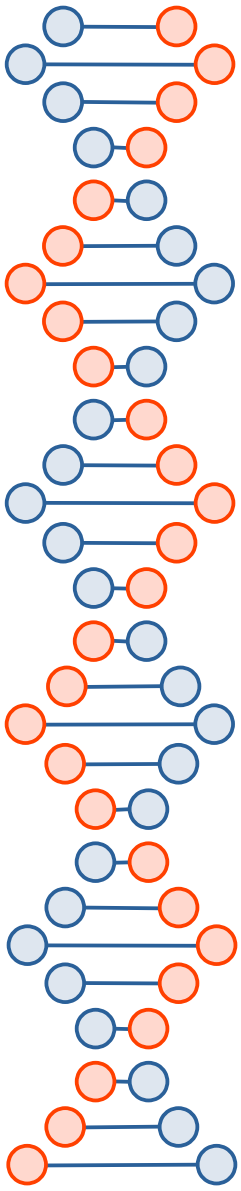
$$\text{Recall} = \frac{tp}{tp + fn}$$

sensitivity = recall

$$F = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

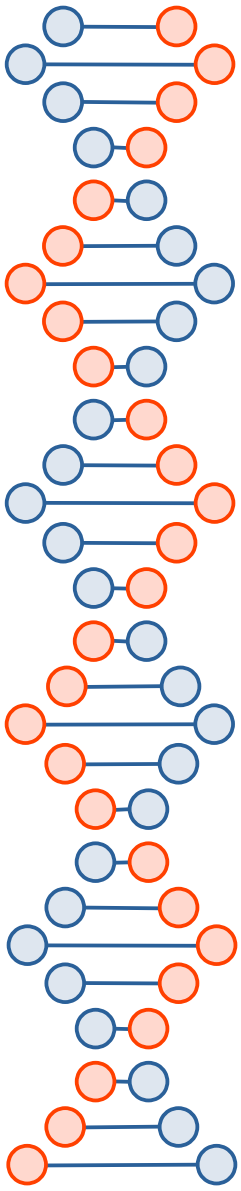
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$



Classification Accuracy (1)

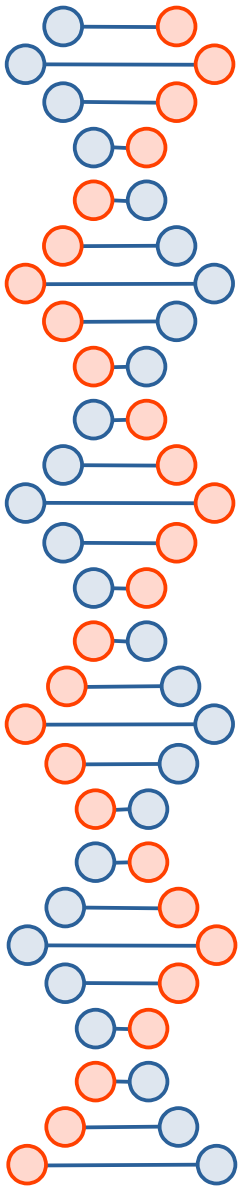
- A quick way to evaluate a set of predictions on a classification problem is by using accuracy.
- Classification accuracy is a ratio of the number of correct predictions out of all predictions that were made.
- It is often presented as a percentage between 0% for the worst possible accuracy and 100% for the best possible accuracy.

Accuracy = correct predictions × 100 / total predictions



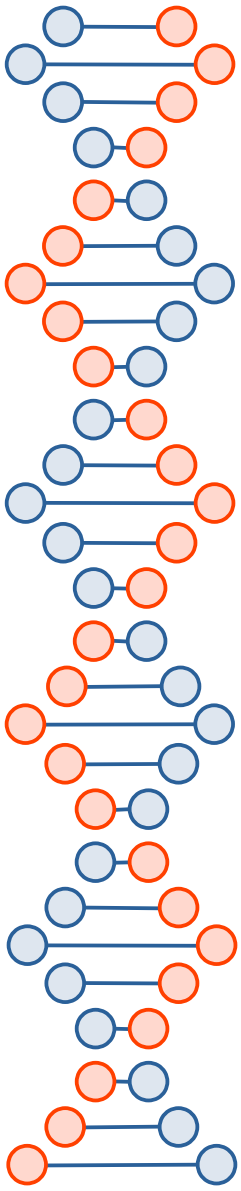
Classification Accuracy (2)

- We can implement this in a function that takes the expected outcomes and the predictions as arguments.
- Below is this function named accuracy_metric() that returns classification accuracy as a percentage.
- Notice that we use == to compare the numeric equality actual to predicted values.
- This allows us to compare integers or strings, two main data types that we may choose to use when loading classification data.



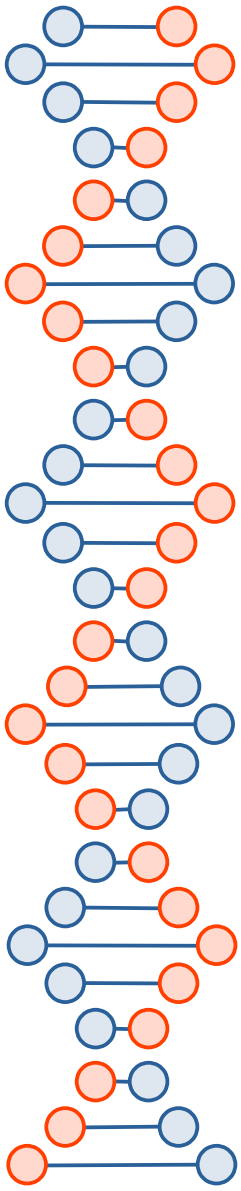
Classification Accuracy (3)

```
# Calculate accuracy percentage between two lists
use strict;
use warnings;
use Data::Dump qw(dump);
use List::Util qw(zip min max sum);

 # Defined in Section 4.2.1 Classification Accuracy
 # Function To Calculate Classification Accuracy.
 # Calculate accuracy percentage between two lists
 sub accuracy_metric{
   my ($actual, $predicted) = @_;
   my $correct = 0;
   for my $i (0 .. $#$actual){
     if ($actual->[$i] == $predicted->[$i]){
       $correct += 1;
     }
   }
   return sprintf '%0.1f', $correct / @$actual * 100.0;
 }

# Test accuracy
my $actual    = [0,0,0,0,0,1,1,1,1,1];
my $predicted = [0,1,0,0,0,1,0,1,1,1];
print "actual\tpredicted\n";
printf  "%s\t%s\n" $_->[0], $_->[1] for zip ($actual, $predicted);
my $accuracy  = accuracy_metric($actual, $predicted);
print $accuracy;
# Example Output From Calculating Classification Accuracy.
# actual predicted
# 0   0
# 0   1
# 0   0
# 0   0
# 0   0
# 1   1
# 1   0
# 1   1
# 1   1
# 1   1
# 80.0
```



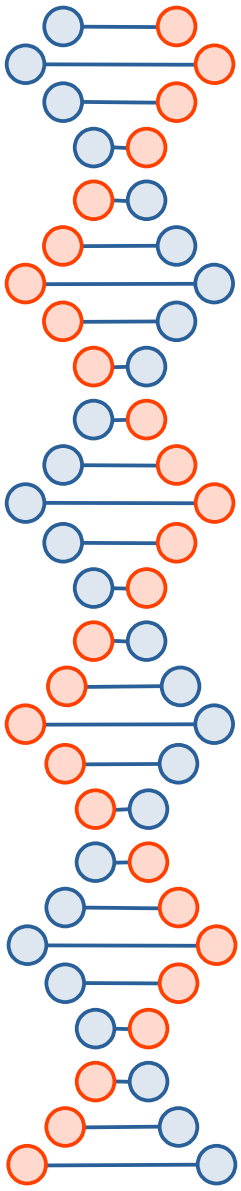
Classification Accuracy (4)

- Accuracy is a good metric to use when you have a small number of class values, such as 2, also called a binary classification problem.
- Accuracy starts to lose it's meaning when you have more class values and you may need to review a different perspective on the results, such as a confusion matrix.



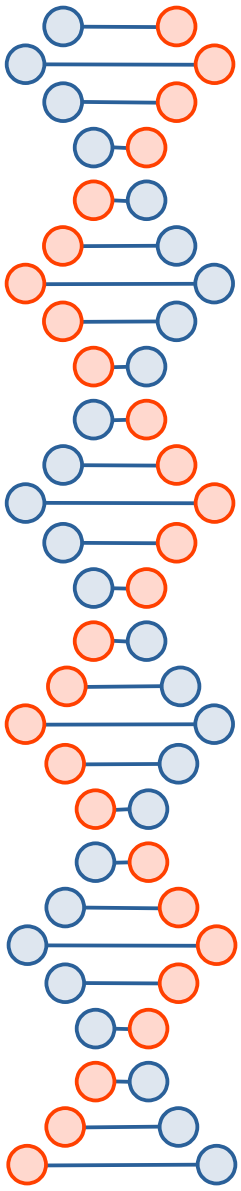
Confusion Matrix (1)

- A confusion matrix provides a summary of all of the predictions made compared to the expected actual values.
- The results are presented in a matrix with counts in each cell.
- The counts of predicted class values are summarized horizontally (rows), whereas the counts of actual values for each class values are presented vertically (columns).
- A perfect set of predictions is shown as a diagonal line from the top left to the bottom right of the matrix.
- The value of a confusion matrix for classification problems is that you can clearly see which predictions were wrong and the type of mistake that was made.



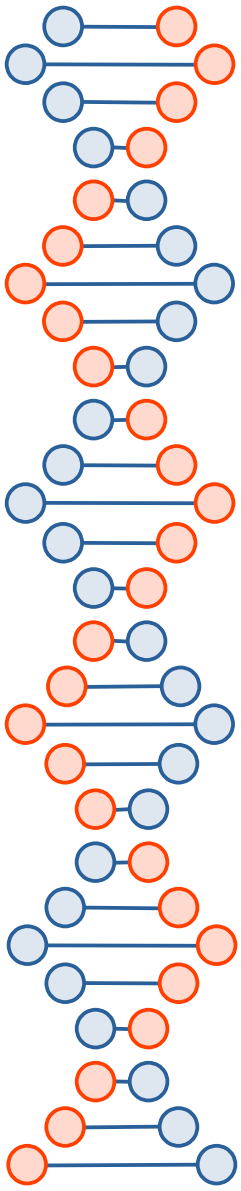
Confusion Matrix (2)

- Let’s create a function to calculate a confusion matrix.
- We can start off by defining the function to calculate the confusion matrix given a list of actual class values and a list of predictions.
- The function is listed below and is named confusion_matrix().
- It first makes a list of all of the unique class values and assigns each class value a unique integer or index into the confusion matrix.
- The confusion matrix is always square, with the number of class values indicating the number of rows and columns required.
- Here, the first index into the matrix is the row for actual values and the second is the column for predicted values.
- After the square confusion matrix is created and initialized to zero counts in each cell, it is a matter of looping through all predictions and incrementing the count in each cell.
- The function returns two objects. The first is the set of unique class values, so that they can be displayed when the confusion matrix is drawn. The second is the confusion matrix itself with the counts in each cell.



Confusion Matrix (3)

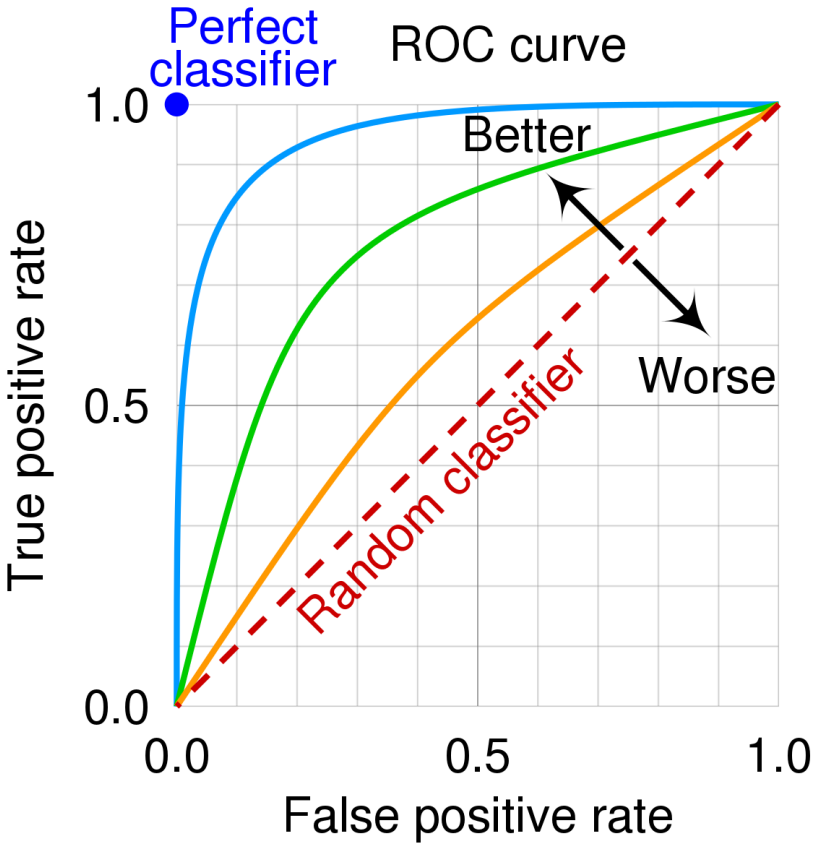
```
# Example of Calculating and Displaying a Pretty Confusion Matrix
 # Defined in Section 4.2.2 Confusion Matrix
 # Function To Calculate a Confusion Matrix.
 # calculate a confusion matrix
 sub confusion_matrix{
   my ($actual, $predicted) = @_;
   tie my %unique, 'Tie::IxHash'; # Order preserved for building @unique, which is later used for printing the confusion matrix
   my @unique = grep{defined $_} %unique = map {$_ => undef} @$actual;
   my $matrix = [map {[]} 0 .. $#unique];
   for my $i (0 .. $#unique){
     $matrix->[$i] = [0, map {$_} 0 .. $#unique -1];
   }
   my %lookup = ();
   while (my ($i, $value) = each @unique) {
     $lookup{$value} = $i;
   }
   for my $i (0 .. $#$actual){
     my $x = $lookup{$actual->[$i]};
     my $y = $lookup{$predicted->[$i]};
     $matrix->[$y][$x] += 1;
   }
   return \@unique, $matrix;
 }

# Function To Pretty Print a Confusion Matrix.
# pretty print a confusion matrix
sub print_confusion_matrix{
  my ($unique, $matrix) = @_;
  print '(A) ' . join(' ', map {$_} @$unique), "\n";
  print "(P)--- \n";
  while (my ($i, $x) = each @$unique){
    print sprintf " %s| %s\n", $x, join(' ', map {$_} @{$matrix->[$i]});
  }
}

# Example Output From Printing a Pretty Confusion Matrix.
# Test confusion matrix with integers
my $actual = [0,0,0,0,0,1,1,1,1,1];
my $predicted = [0,1,1,0,0,1,0,1,1,1];
my ($unique, $matrix) = confusion_matrix($actual, $predicted);
print_confusion_matrix($unique, $matrix);
# (A)0 1 # Example Output From Printing a Pretty Confusion Matrix.
# (P)---
# 0| 3 1
# 1| 2 4
```

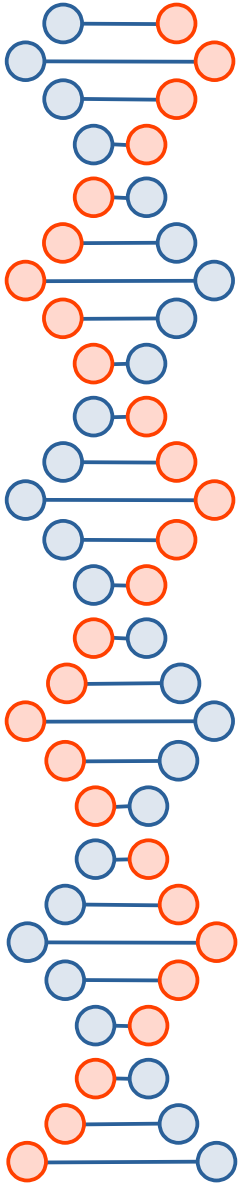


ROC curves (1)

- A receiver operating characteristic curve, or ROC curve, is a graphical plot that illustrates the performance of a binary classifier model at varying threshold values.
- The ROC curve is the plot of the true positive rate against the false positive rate at each threshold setting.
- The diagonal of an ROC graph can be interpreted how to guess at random, and the classification models that are located below the diagonal are considered worse than random guesses.

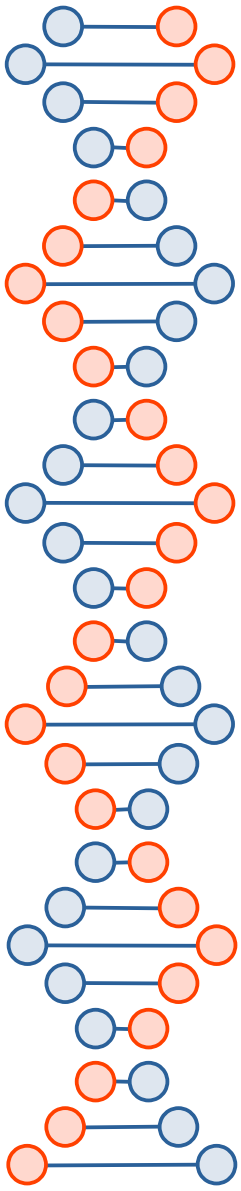
ROC curves (2)


Perfect classifier
ROC curve
1.0
Better
Worse
0.5
Random classifier
True positive rate
0.0
0.0
0.5
1.0
False positive rate



ROC curves (3)

- A perfect classifier would land in the top corner left of the graph with a true positive rate of 1 and a false positive rate of 0.
- Based on the ROC curve, we can calculate the area under the curve (AUC) to characterize the performance of a classification model.



Summary

- In this tutorial, you discovered how to implement algorithm prediction performance metrics from scratch. Specifically, you learned:

•How to measure classification results.

- How to implement and interpret classification accuracy.
- How to implement and interpret the confusion matrix for classification problems.

•How to interpret a ROC curve.